

Zero-TTT

单卡、顺序执行的 AlphaZero 与测试时训练融合规划

个人研究与娱乐项目

2026 年 8 月

摘要

本文规划一个面向 19 路围棋的 AlphaZero 与 Test-Time Training (TTT) 融合机制原型。项目以单进程、单张不超过 16 GiB 的 NVIDIA GPU 和所有模型调用 batch size 固定为 1 为硬约束，优先验证规则、搜索、快权重生命周期、元梯度和恢复闭环，而不承诺棋力提升或训练效率。模型在最后两个 Transformer 块的 FFN down-projection 上叠加可更新残差：长期残差分别由黑白双方在整盘棋中持有，短期残差由当前行棋方在每手 warm MCTS 后临时更新。短期单步更新保留精确二阶元梯度；长期更新采用带动量与向初始化遗忘的一阶直通近似，不保存跨回合高阶计算图。训练采用同一模型快照的两遍顺序流程：第一遍无梯度完成整盘自博弈并持久化搜索轨迹，终局已知后第二遍从开局重建快状态、逐手反向，最后只执行一次外循环优化器更新。

	19×19，贴目 7.5，面积计分，362 个动作
棋盘与规则	
	单进程、单 GPU、单局同步；无异步 actor/learner
执行拓扑	
	所有模型前向与反向的 batch size 固定为 1
模型批量	
	6 块，隐藏维度 192，6 头，FFN 维度 768
默认模型	
	每手 warm 32 次，更新短期状态后 final 96 次
搜索预算	
	一阶直通近似；数值状态整盘保留，计算图逐手释放
长期元梯度	
	每手一次可微 SGD，精确传播二阶梯度
短期元梯度	
	FP32、PyTorch eager、非重入 activation checkpoint
数值与执行	
	默认配置峰值低于 15 GiB，给 16 GiB 单卡留出余量
显存目标	

目录

1	项目定位与可行性边界	3
1.1	完成目标	3
1.2	严格的非并行约束	3
1.3	理论判断与论文取舍	3
2	固定围棋问题与模型接口	4
2.1	规则环境	4
2.2	输入、角色和输出	4
3	参考模型与残差快权重	5
3.1	默认配置	5
3.2	有效 down-projection	5
3.3	运行时状态与生命周期	6
4	快权重写入规则	6
4.1	搜索节点目标	6
4.2	短期单步精确二阶更新	6
4.3	长期动量、遗忘与一阶近似	7
4.4	隐私与版本边界	7
5	两阶段搜索与同快照两遍训练	7
5.1	真实回合协议	7
5.2	第一遍：无梯度自博弈	8
5.3	第二遍：顺序可微回放	8
6	搜索档位与计算预算	9
7	监督预训练与数据边界	9
8	显存、数值与恢复策略	10
8.1	显存控制	10
8.2	检查点和故障恢复	10
9	工程组织与固定环境	10
10	实施冲刺	11
11	测试与验收标准	11
11.1	规则、数据和静态模型	11
11.2	状态、更新和元梯度	12
11.3	搜索、两遍一致性和恢复	12
12	风险与明确不做的事项	12
13	推荐实施顺序	13

1 项目定位与可行性边界

1.1 完成目标

Zero-TTT 的完成条件是一个可检查、可恢复的机制闭环：

- 在固定的 19 路规则环境上运行策略-价值网络和顺序 PUCT；
- 在单手内从 warm 搜索监督更新短期快权重，再用新状态重新搜索；
- 在整盘棋中维护黑白两份相互隔离的长期数值状态，并在每个真实动作提交后更新；
- 在终局已知后回放同一轨迹，使短期初始化获得精确二阶信用，使长期初始化获得一阶直通信用；
- 支持静态监督预训练、完整自博弈、轨迹持久化、整盘学习、崩溃恢复和 Docker 迁移；
- 用小模型和明确预算在 16 GiB 单卡上跑通，而不是复现 AlphaGo Zero 的棋力或吞吐率。

1.2 严格的非并行约束

MVP 明确不实现并行自博弈、并行棋局、批量叶节点推理、树并行、序列并行、上下文并行、数据并行、DDP、FSDP、流水线并行、张量并行、跨设备快状态或异步 actor/learner。任意时刻只有一盘活动棋局、一套黑白长期状态和至多一份当前手短期状态。

batch size 为 1 的约束覆盖监督预训练、自博弈网络评价、warm trace 写入、final root 评价和学习回放。允许把多个 batch-size-1 损失顺序累积到同一个优化器步骤，也允许 CUDA 在线性层和注意力矩阵内部使用普通硬件并行；这两者都不构成多样本或多序列训练。

该边界有意牺牲吞吐率，以换取直接的 Python 控制流、动态长度轨迹、逐手动量更新、确定性回放和较小的状态管理面。原始 TTT 的 mini-batch dual form、LaCT 的大 chunk 和 In-Place TTT 的并行 scan 都不在 MVP 中实现[1, 5, 6]。

1.3 理论判断与论文取舍

原始 TTT 把可学习模型的参数作为 RNN 隐状态，并以梯度下降写入上下文[1]。TTT-E2E 进一步直接使用网络末端任务损失更新后段 MLP，并通过外循环优化其初始化；其论文和官方实现都选择只更新后四分之一块、使用 SGD 和内循环梯度裁剪[2]。本项目沿用这一“任务损失写入、外循环训练初始化”的原则，但时间轴是围棋真实回合和 warm 搜索节点，不是语言 token。

In-Place TTT 表明只更新既有 FFN 的 down-projection 可以显著缩小快状态范围[6]。本项目采用更保守的残差形式：静态 down-projection 不变，快状态只表示附加的 ΔW 。Titans 的动量 surprise 与遗忘机制为长期更新提供直接依据[3]；ATLAS 的 Omega 规则和 Muon、LaCT 的归一化与 Muon 说明了更复杂更新器的潜力[4, 5]，但它们会增加状态、矩阵迭代和高阶求导风险，因此只列为未来方向。

MCTS 是离散策略改进器，本项目不对选择、扩展、节点筛选、访问计数或动作采样求导。所有搜索统计均为停止梯度目标。因此，“端到端”只表示外循环梯度穿过网络及短期快权重更新，不表示穿过搜索算法。

2 固定围棋问题与模型接口

2.1 规则环境

项目只支持 19×19 棋盘、贴目 7.5、面积计分和 361 个落点加停着共 362 个动作。MVP 不支持让子棋、其他贴目、日式数子、其他棋盘尺寸或认输。

规则适配层固定基于 PettingZoo 1.26.1 的 `go_v5` 及其 `Minigo Position`[8, 9]，并对 MCTS 暴露不可变接口：

- 从动作历史重建位置、派生子状态、生成合法动作掩码并提交真实动作；
- 维护当前及过去 7 个局面所需的 17 平面输入；
- 在 softmax 前屏蔽非法动作，并断言停着索引始终为 361；
- 把终局结果转换为每个历史状态当前行棋方视角的 $z_t \in \{-1, +1\}$ 。

2.2 输入、角色和输出

模型输入固定为

$$x_t \in \{0, 1\}^{1 \times 17 \times 19 \times 19}.$$

前 16 个平面表示当前及过去 7 个局面中双方棋子的位置，最后一个平面表示当前行棋方。开局缺失历史使用全零平面填充。另有角色标记

$$r \in \{\text{self}, \text{opponent}\},$$

通过小型嵌入加入全局 token，用来区分从自身搜索策略学习与从对手公开动作学习。

`ModelOutput` 只包含：

- 策略 logits $l_t \in \mathbb{R}^{1 \times 362}$ ，经合法动作掩码后得到 p_t ；
- 标量价值 $v_t \in [-1, 1]^{1 \times 1}$ ，表示当前行棋方的预期终局结果。

MVP 不包含逐动作 Q 预测头。MCTS 边 Q 只存在于搜索树内部，用于 PUCT 备份及构造节点搜索价值，不成为模型公共接口。

3 参考模型与残差快权重

3.1 默认配置

配置项	MVP 默认值
Transformer 块数	6
隐藏维度	192
注意力头数	6
FFN 隐藏维度	768
空间 token	361 个交叉点 + 1 个全局 token
注意力	完整、非因果、标准缩放点积注意力
归一化	LayerNorm
dropout	0
长期快权重块	倒数第 2 个块
短期快权重块	最后 1 个块
参数与计算 dtype	FP32

4 块、隐藏维度 96 的 tiny 配置只用于 CPU 和梯度测试。原先的 8 块、隐藏维度 256、FFN 维度 1024 配置保留为非 MVP 研究档，不能作为首个端到端闭环的验收配置。

3.2 有效 down-projection

设某个 FFN 的静态中间激活为

$$h = \sigma(W_{\text{up}}x + b_{\text{up}}),$$

则两个自适应块的输出都使用

$$\text{FFN}(x) = (W_{\text{static}} + \Delta W_{\text{fast}})h + b_{\text{down}}.$$

注意力、归一化、up-projection、静态 down-projection、嵌入、二维 RoPE 和输出头均为慢权重。内循环白名单只允许替换对应块的 ΔW_{fast} ，从而把每份快状态限制为一个 192×768 矩阵。

系统学习两份残差初始化

$$\theta_{\text{game}} \in \mathbb{R}^{192 \times 768}, \quad \theta_{\text{search}} \in \mathbb{R}^{192 \times 768},$$

二者初始值均为零，但作为外循环参数继续训练。静态矩阵始终保留，因而即使快状态发生漂移，模型仍有不被内循环覆盖的基础路径。

3.3 运行时状态与生命周期

每盘开始时创建

$$\phi_0^B = \text{clone}(\theta_{\text{game}}), \quad \phi_0^W = \text{clone}(\theta_{\text{game}}),$$

并为双方分别创建同形状的零动量 m_0^B, m_0^W 。每手开始时只为当前行棋方创建

$$\psi_{t,0} = \text{clone}(\theta_{\text{search}}).$$

接口	必需字段与生命周期
<code>GameFastState</code>	当前残差、动量、版本号；黑白各一份，整盘持久化。
<code>SearchFastState</code>	当前残差和版本号；每手从 θ_{search} 创建，真实落子后销毁。
<code>ModelOutput</code>	策略 logits 和当前行棋方标量价值。
<code>SearchTrace</code>	根相对动作路径、访问策略、搜索价值、总访问数；纯数据且停止梯度。
<code>GameTrajectory</code>	模型/配置哈希、随机数状态、真实动作、两阶段 trace 和终局结果。

4 快权重写入规则

4.1 搜索节点目标

对任意已搜索节点 u ，由其子边访问计数和备份均值构造

$$\pi_u(a) = \frac{N_u(a)}{\sum_b N_u(b)}, \quad V_u = \sum_a \pi_u(a) Q_u(a).$$

策略目标只覆盖合法动作， V_u 是当前节点行棋方视角。warm 写入集固定为

$$\mathcal{D}_t^{\text{warm}} = \{\text{root}\} \cup \text{Top4}\{u : N_u \geq 4, u \neq \text{root}\}.$$

内部节点按总访问数排序；数据不足时自然退化为只使用根节点。节点只以根相对动作路径保存，回放时由规则环境重建，不序列化 Python 棋盘对象。

4.2 短期单步精确二阶更新

设 ρ 为 Huber 损失，节点权重为

$$\omega_u = \frac{N_u}{\sum_{v \in \mathcal{D}_t^{\text{warm}}} N_v}.$$

短期写入损失为

$$\ell_t^{\text{search}} = \sum_{u \in \mathcal{D}_t^{\text{warm}}} \omega_u [\text{CE}(\pi_u, p_u) + \lambda_s \rho(v_u - V_u)],$$

其中 MVP 固定 $\lambda_s = 1$ 。所有节点逐个执行 batch-size-1 前向，损失汇总后只更新一次：

$$g_s = \nabla_{\psi_{t,0}} \ell_t^{\text{search}}, \quad \psi_{t,1} = \psi_{t,0} - 0.1 \text{ clip}_{1.0}(g_s).$$

学习回放时为该梯度继续构造计算图，使适应后外层损失能向 θ_{search} 和相关慢权重传播精确二阶梯度。自博弈第一遍使用同一数值公式，但不保留梯度图。

4.3 长期动量、遗忘与一阶近似

行棋方长期写入损失使用 final root：

$$\ell_t^{\text{game,self}} = \text{CE}(\pi_t^F, p_t^0) + \rho(v_t^0 - V_t^F),$$

其中 (p_t^0, v_t^0) 由长期状态 ϕ_t^c 和未适应的短期初始化 $\psi_{t,0}$ 在真实根局面上产生。观察方不能读取搜索统计，只使用公开动作

$$\ell_t^{\text{obs}} = \text{CE}(\text{onehot}(a_t), p(s_t; \phi_t^c, \psi_{t,0}, r = \text{opponent})).$$

对两类长期损失分别计算 $g = \nabla_{\phi} \ell$ ，并使用固定更新器

$$m' = 0.9m + 0.1 \text{ stopgrad}(\text{clip}_{1.0}(g)),$$

$$\phi' = \theta_{\text{game}} + 0.999(\phi - \theta_{\text{game}}) - 0.01m'.$$

动量保存近期连续纠偏，0.999 的保留系数使状态缓慢回到可训练初始化，而不是衰减到零。两者都是明确的工程初值，不声称对围棋最优，MVP 不执行超参数搜索。

学习回放在每手使用直通重锚定

$$\phi_{\text{meta}} = \theta_{\text{game}} + \text{stopgrad}(\phi_{\text{runtime}} - \theta_{\text{game}}).$$

其前向值等于当前运行时状态，反向则把当前外层损失直接路由到 θ_{game} ，忽略长期更新梯度的 Hessian 和跨回合 Jacobian。每次长期写入都设置 `create_graph=False` 并立即 detach，因此数值记忆跨整盘保留，但 autograd 图不跨真实回合。

4.4 隐私与版本边界

黑方搜索只能读取 ϕ^B ，白方搜索只能读取 ϕ^W 。一方的 warm/final 策略、价值、树统计和短期梯度不得成为另一方的输入；观察方只接收公开局面和实际动作。每次短期或长期状态变化都递增版本号，并立即使旧树失效。这里的隐私是算法内部的信息隔离，而不是密码学安全。

5 两阶段搜索与同快照两遍训练

5.1 真实回合协议

设第 t 手当前行棋方为 c ，真实落子前状态为 s_t ：

1. 从 θ_{search} 创建 $\psi_{t,0}$, 固定 $(\phi_t^c, \psi_{t,0})$ 顺序执行 32 次 warm PUCT; warm 根不加 Dirichlet 噪声。
2. 导出根加 Top-4 的只读 `SearchTrace`, 汇总短期损失并得到 $\psi_{t,1}$ 。
3. 短期版本变化后清空 warm tree; 不得复用旧先验、叶值、访问数或边 Q。
4. 固定 $(\phi_t^c, \psi_{t,1})$ 从空树执行 96 次 final PUCT。自博弈 final 根加入比例 0.25、 $\alpha = 0.03$ 的 Dirichlet 噪声; 前 30 手按访问分布采样, 之后选择最大访问动作。
5. 规则环境验证并提交 a_t 。提交成功后, 使用未适应短期初始化更新行棋方长期状态, 并用公开动作更新观察方长期状态。
6. 保存本手两份 trace、动作、随机数边界和更新后状态哈希; 销毁 $\psi_{t,1}$ 及全部搜索树。

MCTS 只消费策略 prior 和标量价值。所有叶节点逐个 batch=1 评价, 不使用批量叶合并、虚拟损失或树并行。非法动作先验为零, 价值备份时必须翻转行棋方视角。

5.2 第一遍：无梯度自博弈

一盘自博弈从固定模型快照开始, 直到终局前慢权重和两类初始化都不得改变。模型评价使用 eager mode 和 FP32; 所有快状态更新只保留数值, 不构造外层图。每个完整真实回合结束后原子保存未完成轨迹快照, 内容包括动作历史、两份长期残差与动量、版本、随机数状态和已经产生的 trace。

终局后把结果写入 `GameTrajectory` 并冻结该文件。轨迹必须携带模型参数哈希和配置哈希; 哈希不一致时学习器拒绝回放, 避免在已经变化的初始化上使用声称“同快照”的状态。

5.3 第二遍：顺序可微回放

学习器在外循环更新前从开局和同一模型快照重建规则状态及黑白长期状态。对每个真实回合:

1. 用动作路径重建 warm 节点, 顺序计算短期写入损失并执行一次精确二阶更新;
2. 用适应后的 $\psi_{t,1}$ 在真实根局面评价 (p_t^+, v_t^+) , 但不重跑 MCTS;
3. 以存储的 final 策略和终局结果计算

$$\mathcal{L}_t = \text{CE}(\pi_t^F, p_t^+) + (z_t - v_t^+)^2;$$

4. 对 $\mathcal{L}_t/T$ 立即反向并释放本手短期图, 其中 T 为该局真实手数;
5. 用停止梯度的 final root 和实际动作重建两份长期数值更新, 并核对与第一遍保存的状态哈希。

整盘只累积参数梯度, 不改变模型。全部回合通过后, 外循环 AdamW 执行一次参数更新和一次全局梯度裁剪。权重衰减只由 AdamW 实现, 不重复加入 $\mathcal{L}_t$ 。MVP 不使用 replay buffer, 也不跨多盘累积外循环梯度。

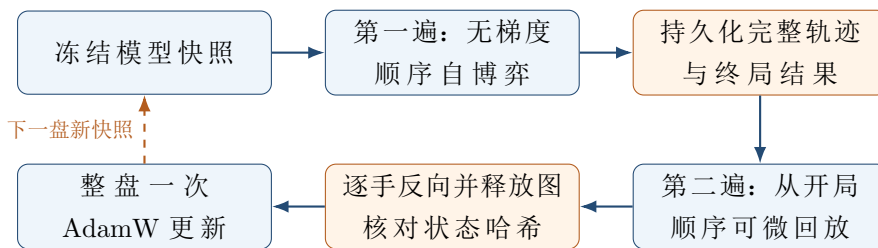


图 1: 同一模型快照的两遍顺序训练。搜索只在第一遍执行，第二遍复用停止梯度轨迹。

6 搜索档位与计算预算

档位	模型	warm	final	用途
tiny/debug	4×96	4	8	CPU 测试、规则测试和恢复冒烟；不评价监督质量。
MVP	6×192	32	96	单卡完整机制闭环和正式验收。
研究档	8×256	64	256	MVP 完成后的人工实验；不进入默认命令。

原方案的 800 次 final 搜索不再出现于 MVP 配置。所有档位的 warm 预算都是额外开销，不从 final 预算中扣除。日志记录每手实际模拟数、网络评价数、搜索耗时、短期写入耗时和回放耗时，但这些性能值不作为完成门槛。

7 监督预训练与数据边界

监督阶段只训练静态策略-价值基线，不执行短期或长期内循环， θ_{game} 与 θ_{search} 保持零并冻结。这样先得到可用的标准 AlphaZero 网络，再单独调试 TTT 闭环，避免把数据解析、在线状态和高阶梯度同时引入。

数据优先取 KataGo g104 的小规模 SGF 子集[10, 11]。MVP 预处理步骤固定为：

1. 过滤 19×19、面积计分、普通自博弈棋局，并记录来源、大小、校验和和 CC0 许可证；
2. 从动作序列重建 17 平面输入、合法掩码和当前行棋方视角终局结果；
3. 默认以实际动作 one-hot 作为策略目标；只有来源字段和语义经过测试确认时才使用软访问分布；
4. 对整盘棋统一采样同一个八重对称变换，保持局面、动作和合法掩码一致；
5. 每次只读取一个位置并执行 batch-size-1 前向，允许顺序累积 32 个位置后更新一次静态模型。

缺失逐动作搜索 Q 不再需要特殊处理，因为 MVP 没有 Q 头。监督预训练完成后冻结一个 baseline 检查点；TTT 自博弈阶段从该检查点启用两份零初始化残差并开始外循环学习。

8 显存、数值与恢复策略

8.1 显存控制

短期更新是唯一使用 `create_graph=True` 的路径，并且只覆盖当前手。Transformer 块在学习回放中使用 `torch.utils.checkpoint.checkpoint(..., use_reentrant=False)`，以支持 `autograd.grad` 和嵌套参数结构[14, 13]。dropout 固定为零，检查点区域不得读取会在重算时变化的全局状态。

默认禁用混合精度、`torch.compile`、TorchScript、自定义 CUDA kernel 和图捕获。参数、快状态、动量、损失和梯度均为 FP32。每个正式命令在开始前重置 CUDA 峰值显存统计，结束后读取 `max_memory_allocated`；MVP 配置要求峰值低于 15 GiB。

8.2 检查点和故障恢复

模型检查点只在整盘回放成功且 AdamW 步骤完成后保存，包含慢权重、两类初始化、优化器、调度器、全局步数、配置/代码/数据哈希及 Python、NumPy、CPU、CUDA 随机数状态。

未完成自博弈在每个完整回合后原子保存动作历史、双方长期残差与动量、版本、随机数状态和已完成 trace。短期状态和 MCTS 树不进入快照；恢复时从当前回合开头重跑 warm/final 搜索。

学习回放不保存部分 autograd 图，也不在中途更新模型。若回放崩溃，直接从已完成的棋局轨迹开头重跑；由于模型尚未更新，这一选择比序列化高阶图更简单且确定。

9 工程组织与固定环境

建议实现模块边界如下：

子系统	职责
规则适配层	不可变 19 路状态、17 平面历史、合法动作、坐标/对称转换和终局视角
模型层	6 块空间 Transformer、策略/价值头、两个残差 down-projection 插槽和函数式前向
记忆控制器	黑白长期残差与动量、每手短期残差、白名单、裁剪、版本和直通重锚定
MCTS	顺序 PUCT、32/96 两阶段搜索、根噪声、trace 导出和强制清树
轨迹层	动作路径形式的 <code>SearchTrace</code> 、 <code>GameTrajectory</code> 、哈希和原子写入

学习器	同快照回放、短期二阶、长期一阶直通、逐手反向和整盘一次 AdamW
数据层	g104 SGF 小子集、17 平面重建、对称增强和 batch=1 流式读取
恢复层	完整回合快照、整盘模型检查点、随机数恢复和确定性重放

参考环境固定为 Linux Docker、Python 3.12、PyTorch 2.11.0 的 CUDA 12.8 wheel[12]、PettingZoo 1.26.1[9]。依赖通过 `pyproject.toml` 和 `uv.lock` 精确锁定，数据、轨迹、日志和检查点均通过 volume 挂载。CPU 只要求运行规则、数据、tiny 模型和小规模高阶梯度测试。

10 实施冲刺

冲刺	主题	完成定义
0	规则与环境	锁定依赖；完成 19 路状态、17 平面、合法掩码、对称、终局视角及 tiny CPU 测试。
1	静态基线	完成 6×192 策略-价值网络、g104 SGF 小子集、batch=1 监督预训练和 checkpoint。
2	顺序 PUCT	完成无 TTT 的 debug/MVP 搜索、一盘 19 路对局、根噪声、动作采样和树版本测试。
3	快状态	完成残差 down-projection、短期单步二阶、长期动量/遗忘、一阶直通和黑白隔离。
4	两遍闭环	完成 32+96 自博弈、可序列化 trace、同快照回放、状态哈希一致和整盘一次 AdamW。
5	恢复与容器	完成回合边界恢复、回放重启、Docker CPU/GPU 冒烟及低于 15 GiB 的显存验收。
未来	非 MVP	Q 头、800 搜索、精确长期二阶、完整自适应 FFN、Muon/Omega、低秩和所有并行方案。

11 测试与验收标准

11.1 规则、数据和静态模型

- 固定棋谱覆盖单子/多子提取、打劫、禁入点、停着、连续双停和面积计分；
- 动作编号、SGF 坐标和八重对称可逆，17 平面在提子、停着和开局补零时正确；
- 所有入口断言 batch 维度为 1，模型只返回策略 logits 和标量价值；

- 静态监督预训练期间两份快权重初始化保持零且无梯度。

11.2 状态、更新和元梯度

- ϕ_0^B 与 ϕ_0^W 数值相同但存储独立；更新任一方不会改变另一方；
- warm 后只改变短期残差；下一手 $\psi_{t,0}$ 再次等于 θ_{search} ；
- 长期更新只改变目标玩家的残差、动量和版本，不改变慢权重或短期状态；
- 改变对方私有 warm/final trace 不影响观察方更新；
- 有限差分 and tiny 模型验证短期外层损失对 θ_{search} 的二阶元梯度有限且非零；
- 验证长期外层损失对 θ_{game} 的直通梯度有限且非零，并确认长期状态没有跨回合 `grad_fn`；
- activation checkpoint 开关前后的输出、一阶梯度和短期二阶梯度在规定容差内一致。

11.3 搜索、两遍一致性和恢复

- warm 与 final 搜索内权重版本固定；任何状态更新后旧树均为空；
- warm trace 只有根或不足 4 个内部节点时仍可完成一次归一化短期更新；
- 完成一盘 19 路 debug 对局和一盘默认 32+96 对局，不使用批量叶评价；
- 对同一快照，第一遍与第二遍每手长期残差、动量及版本哈希一致；
- 完整回放能产生有限外层损失并只在整盘结束执行一次 AdamW；
- 回合快照恢复后从当前手开头重跑搜索，结果与相同随机状态的无中断运行一致；
- 回放中断后从完整轨迹开头重跑，模型更新结果与无中断回放一致；
- 默认配置 `max_memory_allocated` 低于 15 GiB，Docker CPU/GPU 冒烟通过。

12 风险与明确不做的事项

- **batch=1 吞吐率低**：这是减少状态管理和并行正确性风险的主动选择；耗时只记录，不作为验收失败。
- **两遍流程增加重算**：第二遍不重跑 MCTS，只重建网络前向和快状态；换取终局目标可用、图及时释放和确定恢复。
- **长期梯度是近似**：直通重锚定忽略更新器的 Hessian 和跨手 Jacobian，文档不把它称为整盘精确元梯度。
- **搜索监督可能自我强化**：warm/final 统计全部停止梯度，预测价值不直接初始化边 Q；是否提升棋力保持为开放问题。

- **固定更新器可能不最优**：0.1/0.01 学习率、0.9 动量和 0.999 保留率仅为可复现工程默认值，不安排搜索或消融。
- **规则依赖风险**：固定 PettingZoo 包版本，并用独立棋谱测试覆盖适配层，不把第三方对象直接写入轨迹。

明确不属于 MVP 的内容包括：辅助逐动作 Q 头、800 次搜索、长期精确二阶、完整自适应 FFN、Muon、Newton-Schulz、Omega 规则、数据依赖遗忘、快权重 L2 归一化、低秩压缩、混合精度、模型编译、自定义 kernel、并行棋局、批量叶节点、DDP/FSDP、序列/上下文/张量/流水线并行、异步 actor/learner、replay buffer、自动超参数搜索、消融、性能基准、Elo、前端和后端服务。

13 推荐实施顺序

首先建立不含 TTT 的可靠基线：规则、输入、策略-价值网络、SGF 小子集和顺序 PUCT。随后在 tiny 模型上只实现一个残差 down-projection，分别验证短期精确二阶和长期一阶直通，再扩展到黑白双长期状态及状态版本。

快状态单元测试通过后，先实现第一遍无梯度自博弈和动作路径 trace，再实现不重跑搜索的第二遍回放。两遍逐手状态哈希完全一致之后，才允许接入整盘一次 AdamW、回合快照、Docker 和默认 32+96 档位。研究档模型和所有高级更新器必须等待 MVP 的显存、恢复和确定性验收全部通过。

参考文献

- [1] Yu Sun, Xinhao Li, Karan Dalal, et al. *Learning to (Learn at Test Time): RNNs with Expressive Hidden States*. 项目本地论文: [paper/Learn at Test Time.pdf](#).
- [2] Arnub Tandon, Karan Dalal, Xinhao Li, et al. *End-to-End Test-Time Training for Long Context*. 项目本地论文: [paper/End-to-End Test-Time Training for Long Context.pdf](#).
- [3] Ali Behrouz, Peilin Zhong, Vahab Mirrokni. *Titans: Learning to Memorize at Test Time*. 项目本地论文: [paper/Titans Learning to Memorize at Test Time.pdf](#).
- [4] Ali Behrouz, Zeman Li, Praneeth Kacham, et al. *ATLAS: Learning to Optimally Memorize the Context at Test Time*. 项目本地论文: [paper/ATLAS Learning to Optimally Memorize the Context at Test Time.pdf](#).
- [5] Tianyuan Zhang, Sai Bi, Yicong Hong, et al. *Test-Time Training Done Right*. 项目本地论文: [paper/Test-Time Training Done Right.pdf](#).
- [6] Guhao Feng, Shengjie Luo, Kai Hua, et al. *In-Place Test-Time Training*. 项目本地论文: [paper/In-Place Test-Time Training.pdf](#).
- [7] David Silver, Julian Schrittwieser, Karen Simonyan, et al. Mastering the Game of Go without Human Knowledge. *Nature*, 550:354–359, 2017.

- [8] Farama Foundation. PettingZoo Go Environment, `go-v5`. <https://pettingzoo.farama.org/environments/classic/go/>
- [9] Farama Foundation. PettingZoo 1.26.1 release. <https://pypi.org/project/pettingzoo/>
- [10] KataGo Project. g104 Downloadable Self-Play Data: Format and Contents. <https://katagoarchive.org/g104/README.txt>
- [11] KataGo Project and Jane Street. g104 Data License (CC0). <https://katagoarchive.org/g104/LICENSE.txt>
- [12] PyTorch Foundation. PyTorch 2.11 Release Blog. <https://pytorch.org/blog/pytorch-2-11-release-blog/>
- [13] PyTorch Contributors. `torch.autograd.grad` Documentation. <https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html>
- [14] PyTorch Contributors. Activation Checkpointing Documentation. <https://docs.pytorch.org/docs/stable/checkpoint.html>